MOBILE APP DEVELOPMENT

COMPREHENSIVE VIVA PREPARATION GUIDE

Subject	Mobile App Development (MAD)
Course	B.E. / B.Tech – Information Technology
Framework	Dart Programming + Flutter Framework
Topics Covered	Dart Basics, Flutter Widgets, PWA, Lab Experiments
Experiments	10 Lab Experiments (Exp 1 – Exp 10)
Purpose	Quick Revision & Viva Preparation

Department of Information Technology

Academic Year 2024 – 2025

TABLE OF CONTENTS

Complete Index of Topics & Experiments

PART 1: DART PROGRAMMING

Chapters 1–18: Complete Dart Language Reference

CHAPTER 1

Variables in Dart

1.1 Definition

A **variable** is a named storage location in memory that holds a value. In Dart, variables are strongly typed, and every variable has a type associated with it.

1.2 Types of Variable Declarations

var

Dart infers the type automatically. Once assigned, the type cannot change.

```
var name = 'Flutter'; var age = 21;
```

Explicit Type

You declare the type explicitly.

```
String city = 'Nashik'; int marks = 95; double gpa = 9.5; bool isPassed = true;
```

dynamic

The type can change at runtime.

```
dynamic value = 'Hello'; value = 42; // allowed
```

final

Value set only once at runtime; cannot be changed after initialization.

```
final String name = 'Dart'; // name = 'Flutter'; // Error!
```

const

Compile-time constant. Value must be known at compile time.

```
const double PI = 3.14159; const int MAX = 100;
```

late

Declares a non-nullable variable that is initialized later.

```
late String description; description = 'Initialized later';
```

Key Concepts

- Dart uses type inference with var keyword
- final = set once at runtime | const = compile-time constant
- Null safety: variables cannot be null unless declared with ?

• String? name; // nullable variable

• dynamic allows type to change; avoid using it unnecessarily

• late is used for deferred initialization

■ *Note: Dart has sound null safety. Use ? for nullable types. Use ! to assert non-null.*

1.3 Null Safety

```
String? nullableName; // can be null String name = 'Dart'; // cannot be null
print(nullableName?.length); // safe call print(name.length); // always safe
```

VIVA QUESTIONS & ANSWERS

Q: What is a variable in Dart?

A: A variable is a named memory location that stores a value. In Dart, every variable has a specific type.

Q: What is the difference between final and const?

A: final is set once at runtime; const is a compile-time constant. Both cannot be reassigned after initialization.

Q: What is var in Dart?

A: var lets Dart infer the type automatically. The type is determined at the time of assignment and cannot change.

Q: What is dynamic in Dart?

A: dynamic is a special type that allows a variable to hold values of any type, even after initial assignment.

Q: What is null safety in Dart?

A: Null safety means variables cannot be null unless explicitly declared with ? (e.g., String? name). It prevents null reference errors at compile time.

Q: What is the late keyword?

A: late is used to declare a non-nullable variable that will be initialized later before it is used. If accessed before initialization, a runtime error occurs.

Q: What are the built-in data types in Dart?

A: int, double, String, bool, List, Map, Set, Symbol, and dynamic are the built-in types in Dart.

CHAPTER 2

Operators in Dart

Operators are special symbols used to perform operations on operands (variables or values).

Operator Type	Purpose	Symbols
Arithmetic	Used for mathematical calculations	`+ - * / % ~/ (integer division)`
Relational	Compare two values; return bool	`== != > < >= <=`
Logical	Combine boolean expressions	`&& (AND) || (OR) ! (NOT)`
Assignment	Assign values to variables	`= += -= *= /= %= ??=`
Bitwise	Operate on individual bits	`& | ^ ~ << >>`
Type Test	Check or cast types	`is is! as`
Conditional	Short-hand if-else	`condition ? expr1 : expr2`
Cascade	Perform multiple operations on same object	`.. (double dot)`
Null-aware	Handle null values safely	`?? ?. ?..`

2.1 Code Examples

```
// Arithmetic int a = 10, b = 3; print(a + b); // 13 print(a ~/ b); // 3 (integer division) print(a % b); // 1 (remainder) // Null-aware String? name; String result = name ?? 'Default'; // 'Default' name ??= 'Flutter'; // assign only if null // Type Test var x = 'Hello'; print(x is String); // true print(x is! int); // true // Cascade var list = [] ..add(1) ..add(2) ..add(3); print(list); // [1, 2, 3] // Ternary int age = 20; String status = age >= 18 ? 'Adult' : 'Minor';
```

VIVA QUESTIONS & ANSWERS

Q: What are the null-aware operators in Dart?

A: ?? (if null), ??= (assign if null), ?. (null-safe member access), and ?.[] (null-safe index access).

Q: What is the cascade operator in Dart?

A: The cascade operator (..) allows chaining multiple method calls on the same object without repeating the object name.

Q: Difference between / and ~/ in Dart?

A: / returns a double (e.g., 5/2 = 2.5), while ~/ performs integer division and returns an int (e.g., 5~/2 = 2).

Q: What is the is operator?

A: is checks if a variable is of a specific type (e.g., x is String returns true). is! checks if it is NOT that type.

Q: What is ??= operator?

A: It assigns a value only if the variable is currently null. e.g., name ??= "Flutter" assigns "Flutter" only if name is null.

CHAPTER 3

Comments in Dart

Comments are non-executable lines used to document and explain code. Dart supports three types of comments.

3.1 Types of Comments

Single-Line Comment

```
// This is a single-line comment var x = 10; // inline comment
```

Multi-Line Comment

```
/* This is a multi-line comment in Dart */
```

Documentation Comment

Used to generate API documentation. Uses /// (recommended) or /** */.

```
/// Calculates the area of a circle. /// [radius] is the radius of the circle. double area(double radius) { return 3.14159 * radius * radius; }
```

VIVA QUESTIONS & ANSWERS

Q: What are the types of comments in Dart?

A: Single-line (//), Multi-line (/* */), and Documentation comments (/// or /** */).

Q: What is a documentation comment used for?

A: Documentation comments (///) are used to generate API docs using the dart doc tool. They appear in auto-generated documentation.

Q: Are comments executed by Dart?

A: No. Comments are completely ignored by the Dart compiler during execution.

CHAPTER 4

Records in Dart

Records (introduced in Dart 3.0) are anonymous, immutable aggregate types that allow you to bundle multiple values together without creating a class.

4.1 Creating Records

```
// Positional record var point = (10, 20); print(point.$1); // 10 print(point.$2); // 20
// Named record var person = (name: 'Alice', age: 25); print(person.name); // Alice
print(person.age); // 25
```

4.2 Records as Return Types

```
(String, int) getUserInfo() { return ('Alice', 25); } void main() { var (name, age) =
getUserInfo(); print('$name is $age years old'); }
```

Key Concepts

- Records are immutable – fields cannot be changed after creation
- Records use positional ($1, $2) or named (record.name) access
- Records are value types – equality is based on field values
- Records can be destructured using pattern matching
- Introduced in Dart 3.0 along with patterns and sealed classes

VIVA QUESTIONS & ANSWERS

Q: What are Records in Dart?

A: Records are anonymous, immutable aggregate types that bundle multiple values. They are value-typed and support structural equality.

Q: How are records accessed?

A: Positional records use $1, $2, etc. Named records use .fieldName syntax.

Q: When were Records introduced in Dart?

A: Records were introduced in Dart 3.0 (released 2023).

Q: Are records mutable?

A: No. Records are immutable – once created, their fields cannot be changed.

CHAPTER 5

Collections in Dart

Dart provides built-in collection types for storing groups of values: List, Set, and Map.

5.1 List

An ordered collection that can contain duplicate elements. Similar to arrays.

```
List<int> nums = [1, 2, 3, 4, 5]; nums.add(6); nums.remove(3); print(nums.length);
print(nums[0]); // Fixed-length list var fixed = List.filled(3, 0); // [0, 0, 0]
```

5.2 Set

An unordered collection of unique elements. No duplicates allowed.

```
Set<String> fruits = {'apple', 'banana', 'mango'}; fruits.add('apple'); // ignored –
already exists print(fruits.contains('banana')); // true
```

5.3 Map

A collection of key-value pairs. Keys must be unique.

```
Map<String, int> scores = {'Alice': 95, 'Bob': 87}; scores['Charlie'] = 78;
print(scores['Alice']); // 95 print(scores.keys); // (Alice, Bob, Charlie)
print(scores.values); // (95, 87, 78)
```

5.4 Collection Methods

Method	Applicable To	Description
`.add(x)`	List, Set	Adds element x
`.remove(x)`	List, Set, Map	Removes element x
`.contains(x)`	List, Set, Map	Returns true if x exists
`.length`	All	Returns number of elements
`.isEmpty`	All	Returns true if empty
`.map(fn)`	All	Transforms each element
`.where(fn)`	All	Filters elements by condition
`.toList()`	Iterable	Converts to List
`.toSet()`	Iterable	Converts to Set

VIVA QUESTIONS & ANSWERS

Q: What is the difference between List and Set?

A: List is ordered and allows duplicates. Set is unordered and stores only unique values.

Q: What is a Map in Dart?

A: A Map is a collection of key-value pairs. Each key is unique. Access values using map[key].

Q: How do you iterate over a List?

A: Using for loop: for(var item in list), or forEach: list.forEach((e) => print(e)), or for(int i=0; i<list.length; i++).

Q: What is the difference between const and final for collections?

A: const creates an immutable compile-time constant collection. final makes the variable non-reassignable but the collection contents can still change.

Q: What are spread operators in collections?

A: The ... operator spreads elements of one collection into another. e.g., var all = [...list1, ...list2]; combines two lists.

CHAPTER 6

Functions in Dart

A function is a reusable block of code that performs a specific task. Dart is a true object-oriented language where even functions are objects.

6.1 Function Syntax

```
// Basic function returnType functionName(parameters) { // body return value; } //
Example int add(int a, int b) { return a + b; } print(add(3, 4)); // 7
```

6.2 Types of Functions

Named Parameters (Optional)

```
void greet({String name = 'World', int age = 0}) { print('Hello $name, Age: $age'); }
greet(name: 'Alice', age: 25);
```

Positional Optional Parameters

```
void greet(String name, [int age = 0]) { print('$name is $age years old'); }
greet('Bob'); // Bob is 0 years old greet('Bob', 25); // Bob is 25 years old
```

Arrow Functions (Fat Arrow)

```
int square(int n) => n * n; print(square(5)); // 25
```

Anonymous Functions (Lambda)

```
var multiply = (int a, int b) => a * b; print(multiply(3, 4)); // 12 // Used with list
var nums = [1, 2, 3, 4]; nums.forEach((n) => print(n));
```

Higher-Order Functions

```
// Functions that accept other functions List<int> process(List<int> list, Function fn) {
return list.map((e) => fn(e)).toList(); } var doubled = process([1,2,3], (x) => x * 2);
print(doubled); // [2, 4, 6]
```

Recursive Functions

```
int factorial(int n) { if (n <= 1) return 1; return n * factorial(n - 1); }
print(factorial(5)); // 120
```

VIVA QUESTIONS & ANSWERS

Q: What are named parameters in Dart?

A: Named parameters are optional, surrounded by {}, and referenced by name when calling. They can have default values.

Q: What is an arrow function?

A: An arrow function (=>) is a shorthand for functions with a single expression. e.g., int sq(n) => n*n;

Q: What are anonymous functions in Dart?

A: Anonymous functions (lambdas) have no name and can be stored in variables or passed as arguments.

Q: What is a higher-order function?

A: A higher-order function accepts other functions as parameters or returns a function.

Q: What are the main functional methods on List?

A: map() transforms elements, where() filters elements, reduce() combines elements, any() checks if any element satisfies condition, every() checks all elements.

CHAPTER 7

Loops in Dart

Loops are used to repeatedly execute a block of code until a condition is met. Dart supports multiple loop types.

7.1 for Loop

```
for (int i = 0; i < 5; i++) { print(i); }
```

7.2 for-in Loop

```
var fruits = ['apple', 'banana', 'mango']; for (var fruit in fruits) { print(fruit); }
```

7.3 while Loop

```
int i = 0; while (i < 5) { print(i); i++; }
```

7.4 do-while Loop

Executes the body at least once, then checks the condition.

```
int i = 0; do { print(i); i++; } while (i < 5);
```

7.5 forEach

```
[1, 2, 3].forEach((element) { print(element); });
```

7.6 Break and Continue

```
for (int i = 0; i < 10; i++) { if (i == 5) break; // exit loop if (i % 2 == 0) continue;
// skip even print(i); }
```

VIVA QUESTIONS & ANSWERS

Q: What is the difference between while and do-while?

A: while checks the condition before executing; do-while executes the body first and checks condition after. do-while always runs at least once.

Q: What is break?

A: break exits the current loop immediately. Execution continues after the loop.

Q: What is continue?

A: continue skips the current iteration and moves to the next iteration of the loop.

Q: What is for-in loop used for?

A: for-in is used to iterate over elements of an iterable (List, Set, etc.) without managing an index manually.

Q: How does forEach differ from for loop?

A: forEach is a method on iterables that takes a callback function. It cannot use break or continue, unlike a for loop.

CHAPTER 8

Branches in Dart

Branches (conditional statements) control the flow of execution based on conditions.

8.1 if / else if / else

```
int score = 75; if (score >= 90) { print('Grade A'); } else if (score >= 75) {
print('Grade B'); } else if (score >= 60) { print('Grade C'); } else { print('Fail'); }
```

8.2 switch / case

```
String day = 'Mon'; switch (day) { case 'Mon': print('Monday'); break; case 'Tue':
print('Tuesday'); break; default: print('Other day'); }
```

8.3 Ternary Operator

```
int age = 20; String type = age >= 18 ? 'Adult' : 'Minor';
```

8.4 Pattern Matching (Dart 3.0+)

Dart 3.0 introduced enhanced switch expressions and pattern matching.

```
// Switch expression (Dart 3.0) String result = switch (score) { >= 90 => 'A', >= 75 =>
'B', >= 60 => 'C', _ => 'Fail', };
```

VIVA QUESTIONS & ANSWERS

Q: What is the purpose of switch statement?

A: switch evaluates a variable/expression and compares it against multiple case values. It is cleaner than multiple if-else for discrete values.

Q: What happens if break is missing in switch?

A: Without break, execution "falls through" to the next case. Dart does NOT allow fallthrough unless the case is empty.

Q: What is pattern matching in Dart 3.0?

A: Pattern matching allows switch to match on object shapes, types, and conditions, not just discrete values. It enables switch expressions that return values.

Q: What is the default case in switch?

A: The default case executes when no other case matches. It is similar to the else block in if-else.

CHAPTER 9

Error Handling in Dart

Error handling prevents unexpected program crashes by catching and managing exceptions. Dart uses try-catch-finally blocks.

9.1 try / catch / finally

```
try { int result = 10 ~/ 0; // Throws IntegerDivisionByZeroException print(result); } catch (e) { print('Error: $e'); } finally { print('This always runs'); // Cleanup code }
```

9.2 on keyword – Specific Exception

```
try { var list = [1, 2, 3]; print(list[10]); // RangeError } on RangeError { print('Index out of range!'); } on FormatException catch (e) { print('Format error: $e'); } catch (e, s) { print('Unknown error: $e'); print('Stack trace: $s'); }
```

9.3 throw

```
void checkAge(int age) { if (age < 0) throw ArgumentError('Age cannot be negative'); print('Age is $age'); } try { checkAge(-5); } catch (e) { print(e); // ArgumentError: Age cannot be negative }
```

9.4 Custom Exceptions

```
class AgeException implements Exception { String message; AgeException(this.message); @override String toString() => 'AgeException: $message'; } try { throw AgeException('Age too low'); } catch (e) { print(e); }
```

Common Dart Exceptions

- FormatException – Invalid format (e.g., int.parse on non-integer)
- RangeError – Index out of valid range
- ArgumentError – Invalid argument passed to a function
- StateError – Object in invalid state
- UnsupportedError – Operation not supported
- IntegerDivisionByZeroException – Division by zero

VIVA QUESTIONS & ANSWERS

Q: What is the purpose of try-catch in Dart?

A: try-catch is used to handle exceptions. Code that may throw is placed in try; error handling code is placed in catch.

Q: What is the finally block?

A: The finally block runs whether or not an exception occurred. It is used for cleanup operations like closing files or network connections.

Q: What is the difference between throw and rethrow?

A: throw raises a new exception. rethrow re-raises the current exception in a catch block, preserving the original stack trace.

Q: How do you create a custom exception in Dart?

A: Implement the Exception interface: class MyException implements Exception { String message; ... }

Q: What is the on keyword in catch?

A: on allows catching specific exception types. e.g., on FormatException catches only FormatException errors.

CHAPTER 10

Class, Object & Constructor in Dart

Dart is a fully object-oriented language. Everything in Dart is an object. A class is a blueprint for creating objects.

10.1 Class & Object

```
class Student { String name; int age; double gpa; // Default constructor Student(this.name, this.age, this.gpa); void display() { print('Name: $name, Age: $age, GPA: $gpa'); } } void main() { Student s1 = Student('Alice', 20, 9.5); s1.display(); // Name: Alice, Age: 20, GPA: 9.5 }
```

10.2 Types of Constructors

Default Constructor

```
class Car { String brand; Car(this.brand); // shorthand }
```

Named Constructor

```
class Point { double x, y; Point(this.x, this.y); Point.origin() : x = 0, y = 0; // named constructor } var p = Point.origin(); // x=0, y=0
```

Factory Constructor

```
class Singleton { static final Singleton _instance = Singleton._(); Singleton._(); factory Singleton() => _instance; }
```

Constant Constructor

```
class ImmutablePoint { final int x, y; const ImmutablePoint(this.x, this.y); } const p = ImmutablePoint(1, 2);
```

10.3 Encapsulation & Access Modifiers

Dart uses underscore _ prefix for private members (private to the library, not just the class).

```
class BankAccount { double _balance; // private BankAccount(this._balance); double get balance => _balance; // getter set deposit(double amt) => _balance += amt; // setter }
```

VIVA QUESTIONS & ANSWERS

Q: What is a class in Dart?

A: A class is a blueprint/template that defines the properties (fields) and behaviors (methods) of objects.

Q: What is an object?

A: An object is an instance of a class. It has its own copy of the class fields and can access class methods.

Q: What is a named constructor?

A: A named constructor is an additional constructor with a unique name, used to create objects in different ways. e.g., Point.origin()

Q: What is a factory constructor?

A: A factory constructor can return an existing instance instead of always creating a new one. Used for singleton pattern.

Q: What is a const constructor?

A: A const constructor creates a compile-time constant object. All fields must be final.

Q: How is access control achieved in Dart?

A: By prefixing identifiers with _ (underscore), making them library-private. There are no public/private/protected keywords.

Q: What is this keyword in Dart?

A: this refers to the current instance of the class. Used to resolve name conflicts between fields and parameters.

CHAPTER 11

Methods in Dart

Methods are functions defined inside a class. They define the behavior of objects.

11.1 Types of Methods

```
class Calculator { // Instance method int add(int a, int b) => a + b; // Static method (belongs to class, not instance) static double pi() => 3.14159; // Getter String get description => 'Basic Calculator'; // Setter set value(int v) => _value = v; int _value = 0; // Override toString @override String toString() => 'Calculator Object'; }
```

11.2 Abstract Methods

```
abstract class Shape { double area(); // abstract method – no body double perimeter(); } class Circle extends Shape { double radius; Circle(this.radius); @override double area() => 3.14159 * radius * radius; @override double perimeter() => 2 * 3.14159 * radius; }
```

VIVA QUESTIONS & ANSWERS

Q: What is a static method?

A: A static method belongs to the class itself, not an instance. Called using ClassName.method(). Cannot access instance members.

Q: What is a getter and setter?

A: A getter (get) provides read access to a private field. A setter (set) provides write access. They are accessed like properties.

Q: What is an abstract method?

A: An abstract method has no implementation body. It is defined in an abstract class and must be overridden by subclasses.

Q: What is @override annotation?

A: @override indicates that a method is intentionally overriding a superclass method. It is a best practice for clarity.

CHAPTER 12

Class Extension (Inheritance) in Dart

Inheritance allows a class to reuse properties and methods of another class using the extends keyword.

```
class Animal { String name; Animal(this.name); void speak() => print('$name makes a sound'); } class Dog extends Animal { Dog(String name) : super(name); @override void speak() => print('$name barks: Woof!'); } void main() { Dog d = Dog('Max'); d.speak(); // Max barks: Woof! }
```

Inheritance Key Points

- Use extends for inheritance (single inheritance only in Dart)
- super() calls the parent class constructor
- @override to redefine parent methods
- All Dart classes implicitly extend Object
- Use is keyword to check inheritance: dog is Animal // true

VIVA QUESTIONS & ANSWERS

Q: What is inheritance?

A: Inheritance is an OOP concept where a child class acquires properties and methods from a parent class using extends.

Q: Does Dart support multiple inheritance?

A: No. Dart supports single inheritance. For multiple inheritance-like behavior, use mixins.

Q: What is super keyword?

A: super refers to the parent class. Used to call parent constructors (super(args)) or parent methods (super.methodName()).

CHAPTER 13

Mixins in Dart

Mixins are a way to reuse code across multiple class hierarchies without using inheritance. They provide a form of multiple inheritance.

```
mixin Swimmable { void swim() => print('Swimming...'); } mixin Flyable { void fly() => print('Flying...'); } class Duck extends Animal with Swimmable, Flyable { Duck() : super('Duck'); } void main() { Duck d = Duck(); d.swim(); // Swimming... d.fly(); // Flying... }
```

Mixin Rules

- Use mixin keyword to define a mixin
- Use with keyword to apply a mixin to a class
- Multiple mixins can be applied: class X extends Y with A, B, C
- Mixins cannot have constructors
- on keyword restricts mixin to specific superclasses: mixin M on BaseClass

VIVA QUESTIONS & ANSWERS

Q: What is a mixin in Dart?

A: A mixin is a class-like structure used to share code across multiple classes without using inheritance.

Q: How is mixin applied?

A: Using the with keyword: class Dog extends Animal with Swimmable.

Q: Can mixins have constructors?

A: No. Mixins cannot have constructors.

Q: What is the difference between extends and with?

A: extends establishes inheritance (one class). with applies mixins (multiple allowed) for code reuse.

CHAPTER 14

Enums in Dart

An enum (enumeration) is a special class that represents a fixed set of named constant values.

```
// Simple enum enum Direction { north, south, east, west } // Enhanced enum (Dart 2.17+) enum Planet { mercury(3.7), venus(8.87), earth(9.8); final double gravity; const Planet(this.gravity); void describe() => print('$name has gravity $gravity m/s2'); } void main() { Direction d = Direction.north; print(d.name); // north print(d.index); // 0 Planet.earth.describe(); }
```

VIVA QUESTIONS & ANSWERS

Q: What is an enum?

A: An enum is a special type that represents a fixed set of named constants, like days of the week or directions.

Q: What is the index property of enum?

A: index gives the zero-based position of the enum value in its declaration order.

Q: What are enhanced enums?

A: Enhanced enums (Dart 2.17+) can have fields, methods, and constructors, making them more powerful than simple enums.

Q: How do you use enum in switch?

A: switch (direction) { case Direction.north: ... } – enums work perfectly with switch statements.

CHAPTER 15

Extension Methods in Dart

Extension methods add new functionality to existing classes without modifying them or creating subclasses.

```
// Adding methods to String class extension StringExtension on String { String capitalize() { if (isEmpty) return this; return '${this[0].toUpperCase()}${substring(1)}'; } bool get isPalindrome => this == split('').reversed.join(); } void main() { print('hello'.capitalize()); // Hello print('racecar'.isPalindrome); // true } // Extension on int extension IntExtension on int { bool get isEven => this % 2 == 0; List<int> range() => List.generate(this, (i) => i); }
```

VIVA QUESTIONS & ANSWERS

Q: What are extension methods?

A: Extension methods add new methods to existing types without modifying the original class or creating a subclass.

Q: Why use extension methods?

A: They improve code readability and allow adding domain-specific functionality to built-in types like String, int, List.

Q: Can extension methods access private members?

A: No. Extension methods can only access public members of the extended class.

CHAPTER 16

Callable Objects in Dart

In Dart, you can make class instances callable like functions by implementing the call() method.

```
class Adder { final int value; Adder(this.value); // Makes the object callable int call(int other) => value + other; } void main() { var add5 = Adder(5); print(add5(3)); // 8 – called like a function! print(add5(10)); // 15 }
```

VIVA QUESTIONS & ANSWERS

Q: What is a callable object?

A: An object whose class implements a call() method, allowing instances to be invoked like functions.

Q: Why use callable objects?

A: They allow objects to behave as functions while still maintaining state, useful for closures and function objects with configuration.

CHAPTER 17

Class Modifiers in Dart

Dart 3.0 introduced class modifiers that control how classes can be used (extended, implemented, mixed in).

Modifier	Can Extend?	Can Implement?	Can Mixin?	Purpose
`class`	Yes	Yes	No	Default class
`abstract`	Yes	Yes	No	Cannot be instantiated directly
`base`	Yes	No	No	Must be extended; cannot be implemented
`interface`	No	Yes	No	Can only be implemented
`final`	No	No	No	Cannot be extended or implemented
`sealed`	Yes (same lib)	Yes (same lib)	No	Restricted hierarchy; enables exhaustive switch
`mixin class`	Yes	Yes	Yes	Can be used as mixin or class

VIVA QUESTIONS & ANSWERS

Q: What is a sealed class?

A: A sealed class restricts the class hierarchy. It can only be extended or implemented within the same library, enabling exhaustive switch expressions.

Q: What is an interface class?

A: An interface class can only be implemented (not extended). It enforces a contract without sharing implementation.

Q: What is a final class?

A: A final class cannot be extended or implemented outside its library. It prevents modification of the class hierarchy.

Q: What is an abstract class?

A: An abstract class cannot be instantiated directly. It serves as a blueprint with abstract methods that subclasses must implement.

CHAPTER 18

Concurrency in Dart

Dart handles concurrency using async/await, Future, and Stream. Dart is single-threaded but uses an event loop for asynchronous operations.

Future

A Future represents a value that will be available later (asynchronous computation).

```
Future<String> fetchData() async { await Future.delayed(Duration(seconds: 2)); return 'Data loaded!'; } void main() async { print('Fetching...'); String data = await fetchData(); print(data); // Data loaded! }
```

Stream

A Stream provides a sequence of asynchronous events over time.

```
Stream<int> countStream() async* { for (int i = 1; i <= 5; i++) { await Future.delayed(Duration(seconds: 1)); yield i; } } void main() async { await for (int value in countStream()) { print(value); // 1, 2, 3, 4, 5 (one per second) } }
```

Isolates

Isolates are independent workers with their own memory. Used for CPU-intensive tasks.

```
import 'dart:isolate'; void heavyWork(SendPort sendPort) { int result = 0; for (int i = 0; i < 1000000; i++) result += i; sendPort.send(result); }
```

async/await Keywords

- async – marks a function as asynchronous; it returns a Future
- await – waits for a Future to complete before proceeding
- async* – marks a function as an asynchronous generator (returns Stream)
- yield – emits a value from a generator function
- sync* – marks a function as a synchronous generator (returns Iterable)

VIVA QUESTIONS & ANSWERS

Q: What is a Future in Dart?

A: A Future represents an asynchronous operation that will complete with a value or an error at some point in the future.

Q: What is async/await?

A: async marks a function as asynchronous. await pauses execution until the Future completes, without blocking the UI thread.

Q: What is the difference between Future and Stream?

A: A Future provides a single value asynchronously. A Stream provides multiple values over time.

Q: What are Isolates?

A: Isolates are independent execution threads with separate memory. They communicate via message passing. Used for heavy computations.

Q: Is Dart single-threaded?

A: Yes. Dart runs on a single thread with an event loop. Async operations are handled via the event queue, not parallel threads.

PART 2: FLUTTER FRAMEWORK

Chapters 19–28: Flutter from Basics to Common Widgets

CHAPTER 19

Introduction to Flutter Framework

Flutter is an open-source UI framework created by Google for building natively compiled applications for mobile, web, desktop, and embedded platforms from a single codebase.

19.1 Key Features of Flutter

Flutter Features

- Cross-platform: One codebase for Android, iOS, Web, Windows, macOS, Linux
- Hot Reload: See code changes instantly without restarting the app
- Rich Widget Library: Pre-built Material Design and Cupertino widgets
- High Performance: Compiled to native ARM code, uses Skia/Impeller rendering engine
- Dart Language: Uses Dart – a fast, strongly typed, AOT/JIT compiled language
- Open Source: Free to use with strong community support

19.2 Flutter Architecture

```
Flutter Architecture Layers: [Your App Code] <- Dart code | [Flutter Framework] <- Widgets, Rendering, Animation | [Flutter Engine] <- Skia, Dart Runtime, Platform Channels | [Platform Embedder] <- Android, iOS, Web, Desktop
```

19.3 Flutter vs React Native vs Native

Feature	Flutter	React Native	Native
Language	Dart	JavaScript	Swift/Kotlin
Rendering	Own engine (Skia)	Native components	Native UI
Performance	Very High	High	Highest
Code Sharing	~95%	~85%	0%
Hot Reload	Yes	Yes	Limited

VIVA QUESTIONS & ANSWERS

Q: What is Flutter?

A: Flutter is an open-source UI framework by Google for building cross-platform apps for mobile, web, and desktop from a single Dart codebase.

Q: What language does Flutter use?

A: Flutter uses Dart, a strongly typed, object-oriented language developed by Google.

Q: What is Hot Reload in Flutter?

A: Hot Reload injects updated code into the running Dart VM and refreshes the widget tree, showing changes instantly without losing app state.

Q: What is the rendering engine used by Flutter?

A: Flutter uses the Skia graphics engine (and newer Impeller engine) to render widgets directly, without using native platform UI components.

Q: What are the layers of Flutter architecture?

A: Framework layer (Dart widgets/rendering), Engine layer (C++, Skia, Dart runtime), and Embedder layer (platform-specific).

CHAPTER 20

Hello World & runApp() in Flutter

20.1 Simple Hello World App

```
import 'package:flutter/material.dart'; void main() { runApp(MyApp()); } class MyApp extends StatelessWidget { @override Widget build(BuildContext context) { return MaterialApp( title: 'Hello World', home: Scaffold( appBar: AppBar( title: Text('Hello World App'), ), body: Center( child: Text( 'Hello, World!', style: TextStyle(fontSize: 24), ), ), ), ); } }
```

20.2 Importance of runApp()

runApp() is the entry point of every Flutter application. It takes the root widget and attaches it to the screen.

runApp() Functions

- Inflates the given widget and attaches it to the screen
- Sets up the widget tree starting from the root widget
- Must be called from main() function
- Takes a Widget as parameter (usually MaterialApp)
- Initializes the Flutter binding between Dart and the platform
- Without runApp(), nothing is displayed on screen

20.3 MaterialApp vs CupertinoApp

```
// Material Design (Android style) MaterialApp( title: 'My App', theme: ThemeData(primarySwatch: Colors.blue), home: HomeScreen(), ) // Cupertino (iOS style) CupertinoApp( title: 'My App', home: CupertinoPageScaffold(...), )
```

VIVA QUESTIONS & ANSWERS

Q: What is runApp()?

A: runApp() is the entry point function that inflates the root widget and attaches it to the device screen. It must be called from main().

Q: What does main() do in Flutter?

A: main() is the entry point of the Dart program. In Flutter, it calls runApp() with the root widget.

Q: What is MaterialApp?

A: MaterialApp is the root widget for Material Design apps. It provides theme, navigation, localization, and debugging support.

Q: What is Scaffold?

A: Scaffold provides the basic visual structure for Material Design apps: AppBar, Body, FloatingActionButton, Drawer, BottomNavigationBar, etc.

Q: What is the widget tree?

A: The widget tree is the hierarchical structure of all widgets in the app, starting from the root widget down to the leaf widgets.

CHAPTER 21

Widget Tree & Element Tree

Flutter maintains three synchronized trees: the Widget Tree, the Element Tree, and the RenderObject Tree.

```
Widget Tree Structure: MaterialApp ■■■ Scaffold ■■■ AppBar ■ ■■■ Text ('Title') ■■■ Center ■■■ Column ■■■ Text ('Hello') ■■■ Text ('World')
```

Tree	Contents	Purpose
Widget Tree	Immutable widget objects	Blueprint/Configuration. Rebuilt on every setState()
Element Tree	Mutable element objects	Lifecycle management. Links widgets to render objects
RenderObject Tree	Render objects	Layout and painting. What actually appears on screen

21.1 StatelessWidget vs StatefulWidget

```
// StatelessWidget – no mutable state class MyLabel extends StatelessWidget { @override Widget build(BuildContext context) { return Text('I never change'); } } // StatefulWidget – has mutable state class Counter extends StatefulWidget { @override _CounterState createState() => _CounterState(); } class _CounterState extends State<Counter> { int count = 0; @override Widget build(BuildContext context) { return Column(children: [ Text('Count: $count'), ElevatedButton( onPressed: () => setState(() => count++), child: Text('Increment'), ), ]); } }
```

VIVA QUESTIONS & ANSWERS

Q: What are the three trees in Flutter?

A: Widget Tree (configuration/blueprint), Element Tree (lifecycle management), and RenderObject Tree (actual rendering/painting).

Q: What is a StatelessWidget?

A: A widget that does not have mutable state. Its build() method is called once and only rebuilds when its parent changes.

Q: What is a StatefulWidget?

A: A widget that has mutable state. It has a separate State class and rebuilds whenever setState() is called.

Q: What is setState()?

A: setState() notifies Flutter that the state of a StatefulWidget has changed, triggering a rebuild of the widget.

Q: What is BuildContext?

A: BuildContext is a reference to the widget's location in the widget tree. It is used to access theme, navigation, and other inherited widgets.

CHAPTER 22

Flutter Lifecycle Events

Flutter StatefulWidget goes through a defined lifecycle from creation to disposal.

```
Lifecycle Order: createState() | initState() <- Called once when widget is inserted into tree | didChangeDependencies() <- Called when dependencies change | build() <- Called every time widget needs to rebuild | didUpdateWidget() <- Called when parent rebuilds with new config | build() <- Rebuilds after didUpdateWidget() | deactivate() <- Called when widget is removed from tree | dispose() <- Final cleanup; release resources
```

Method	When Called	Common Use
`createState()`	Once, when widget is created	Creates the State object
`initState()`	Once, first time widget inserts	Init data, listeners, controllers
`didChangeDependencies()`	After initState; when deps change	Access InheritedWidget data
`build()`	Every rebuild	Return widget tree
`didUpdateWidget()`	When parent rebuilds widget	Compare old vs new widget
`setState()`	Manual trigger	Mark state as dirty, schedule rebuild
`deactivate()`	Widget removed from tree	Suspend subscriptions
`dispose()`	Widget permanently removed	Cancel timers, close streams

VIVA QUESTIONS & ANSWERS

Q: What is initState() used for?

A: initState() is called once when the widget is first created. Used for initializing controllers, setting up listeners, and loading initial data.

Q: What is dispose() used for?

A: dispose() is called when the widget is permanently removed. Used to cancel timers, close stream subscriptions, and dispose controllers to prevent memory leaks.

Q: What triggers build() to be called?

A: build() is called on first render and every time setState() is called, or when the parent widget rebuilds.

Q: What is the difference between deactivate() and dispose()?

A: deactivate() is called when a widget is removed from the tree but may be reinserted. dispose() is called when permanently removed.

Q: Why is dispose() important?

A: Without dispose(), resources like AnimationControllers, StreamSubscriptions, and TextEditingControllers would cause memory leaks.

CHAPTER 23–28

Common Flutter Widgets

23. Text & RichText Widget

Text displays a string of text with a single style. RichText allows multiple styles within one text block.

```
// Text Widget Text( 'Hello Flutter!', style: TextStyle( fontSize: 24, fontWeight: FontWeight.bold, color: Colors.blue, fontStyle: FontStyle.italic, ), textAlign: TextAlign.center, overflow: TextOverflow.ellipsis, maxLines: 2, ) // RichText Widget RichText( text: TextSpan( text: 'Hello ', style: TextStyle(color: Colors.black, fontSize: 18), children: [ TextSpan(text: 'Flutter', style: TextStyle(color: Colors.blue, fontWeight: FontWeight.bold)), TextSpan(text: ' World!', style: TextStyle(color: Colors.red)), ], ), )
```

24. Row & Column Widget

Row arranges children horizontally. Column arranges children vertically.

```
// Row Row( mainAxisAlignment: MainAxisAlignment.spaceEvenly, crossAxisAlignment: CrossAxisAlignment.center, children: [ Icon(Icons.star, color: Colors.yellow), Text('Favorite'), Icon(Icons.favorite, color: Colors.red), ], ) // Column Column( mainAxisAlignment: MainAxisAlignment.center, crossAxisAlignment: CrossAxisAlignment.start, children: [ Text('Line 1'), Text('Line 2'), Text('Line 3'), ], )
```

MainAxisAlignment Options

- start – pack children at the start
- end – pack children at the end
- center – center children
- spaceBetween – equal space between children, none at edges
- spaceAround – equal space around each child
- spaceEvenly – equal space everywhere including edges

25. Container Widget

Container is the most versatile layout widget. It can have padding, margin, color, size, decoration, and a single child.

```
Container( width: 200, height: 100, margin: EdgeInsets.all(16), padding: EdgeInsets.symmetric(horizontal: 12, vertical: 8), decoration: BoxDecoration( color: Colors.blue, borderRadius: BorderRadius.circular(12), boxShadow: [ BoxShadow(color: Colors.black26, blurRadius: 8, offset: Offset(2, 4)), ], border: Border.all(color: Colors.white, width: 2), ), child: Text('Styled Box', style: TextStyle(color: Colors.white)), )
```

26. Button Widgets

```
// ElevatedButton (raised) ElevatedButton( onPressed: () => print('Pressed!'), style: ElevatedButton.styleFrom(backgroundColor: Colors.blue), child: Text('Click Me'), ) // TextButton (flat) TextButton( onPressed: () {}, child: Text('Text Button'), ) // OutlinedButton (bordered) OutlinedButton( onPressed: () {}, child: Text('Outlined'), ) // IconButton IconButton( icon: Icon(Icons.favorite, color: Colors.red), onPressed: () {}, ) // FloatingActionButton FloatingActionButton( onPressed: () {}, child: Icon(Icons.add), backgroundColor: Colors.blue, )
```

27. Form Widget

Form widget groups form fields and manages validation. Use GlobalKey to access the form state.

```
final _formKey = GlobalKey<FormState>(); Form( key: _formKey, child: Column( children: [ TextFormField( decoration: InputDecoration(labelText: 'Email'), validator: (value) { if (value == null || value.isEmpty) return 'Required'; if (!value.contains('@')) return 'Invalid email'; return null; // valid }, ), ElevatedButton( onPressed: () { if (_formKey.currentState!.validate()) { print('Form is valid!'); } }, child: Text('Submit'), ), ], ), )
```

28. AppBar & SafeArea Widget

```
// AppBar AppBar( title: Text('My App'), backgroundColor: Colors.blue, elevation: 4.0, leading: Icon(Icons.menu), actions: [ IconButton(icon: Icon(Icons.search), onPressed: () {}), IconButton(icon: Icon(Icons.more_vert), onPressed: () {}), ], ) // SafeArea SafeArea( top: true, bottom: true, child: Column( children: [ Text('This content avoids notches and system UI'), ], ), )
```

VIVA QUESTIONS & ANSWERS

Q: What is a Container widget?

A: Container is a convenience widget combining common painting, positioning, and sizing of a single child. It supports padding, margin, border, color, and shadows.

Q: Difference between Row and Column?

A: Row arranges children horizontally (mainAxis = horizontal). Column arranges children vertically (mainAxis = vertical).

Q: What is TextFormField vs TextField?

A: TextField is a basic input field. TextFormField adds form integration with validation via validator callback and works with Form widget.

Q: What is SafeArea?

A: SafeArea adds padding to prevent widgets from being obscured by system UI elements like notches, status bars, and navigation bars.

Q: What is the difference between padding and margin in Container?

A: padding is inner spacing (between Container border and child). margin is outer spacing (space between Container and surrounding widgets).

Q: What is AppBar?

A: AppBar is the top navigation bar of a Scaffold. It contains a title, leading icon, and action buttons.

Q: What is RichText used for?

A: RichText displays text with multiple styles in a single widget using TextSpan objects. Unlike Text which applies one style, RichText supports mixed formatting.

PART 3: LAB EXPERIMENTS

Experiment 1 through 10 – Complete Documentation

This section contains detailed documentation for all 10 lab experiments including aim, theory, algorithm, source code, expected output, conclusion, and viva questions.

EXPERIMENT 1: Flutter Installation & Setup

Experiment No.	1
Title	Flutter Installation & Environment Setup
Subject	Mobile App Development
Department	Information Technology
Aim	To install Flutter SDK, configure environment variables, and verify the Flutter development env

Theory

Flutter SDK is the core tool for building Flutter applications. It includes the Dart SDK, flutter CLI, and the framework libraries. Setting up the environment correctly is the first step in mobile app development.

System Requirements

- Windows 10/11 or macOS 12+ or Linux (Ubuntu 20.04+)
- Minimum 1.64 GB free disk space (excluding IDE)
- Git for Windows (Windows only)
- PowerShell 5.0 or newer (Windows)
- Android Studio or VS Code (IDE)
- Android SDK for Android development

Algorithm / Steps

- Step 1: Go to flutter.dev > Docs > Get Started > Install > [Your OS]
- Step 2: Download the Flutter SDK zip file and extract it to a suitable directory (e.g., C:\flutter)
- Step 3: Add flutter/bin to the system PATH environment variable
- Step 4: Open terminal/command prompt and run: flutter doctor
- Step 5: Install any missing dependencies reported by flutter doctor (Android Studio, Android SDK, etc.)
- Step 6: Accept Android licenses: flutter doctor --android-licenses
- Step 7: Install VS Code or Android Studio as your IDE
- Step 8: Install Flutter and Dart plugins in the IDE
- Step 9: Create a test project: flutter create my_app
- Step 10: Run the app: cd my_app && flutter run

Key Commands

```
# Check Flutter installation flutter doctor # Create new project flutter create project_name # Run the app flutter run # Build APK flutter build apk # Get packages flutter pub get # List connected devices flutter devices
```

Expected Output

```
Doctor summary (to see all details, run flutter doctor -v): [OK] Flutter (Channel stable, 3.x.x) [OK] Android toolchain - develop for Android devices [OK] Android Studio (version x.x) [OK] VS Code (version x.x) [OK] Connected device (1 available) No issues found!
```

Conclusion

The Flutter SDK was successfully installed and configured. The flutter doctor command confirms all required components are properly set up for mobile application development.

VIVA QUESTIONS & ANSWERS

Q: What is Flutter SDK?

A: Flutter SDK is the toolkit that contains the Dart SDK, Flutter framework, flutter CLI, and tools needed to build, test, and deploy Flutter apps.

Q: What is flutter doctor command?

A: flutter doctor diagnoses the Flutter installation and reports any missing components or configuration issues.

Q: What is pubspec.yaml?

A: pubspec.yaml is the configuration file for a Flutter project. It declares dependencies (packages), assets (images, fonts), and project metadata.

Q: What is the purpose of flutter pub get?

A: flutter pub get downloads all packages listed in pubspec.yaml from pub.dev repository to the local project.

Q: What is an emulator vs physical device?

A: An emulator is a virtual device running on the computer. A physical device is an actual Android/iOS phone. Physical devices offer more accurate testing.

Q: What is the Flutter channel?

A: Flutter has channels: stable (production-ready), beta (tested features), dev (latest features), master (cutting-edge). stable is recommended for development.

EXPERIMENT 2: Flutter Container & Layout Widgets

Experiment No.	2
Title	Flutter Container, GridView, ListView, Stack Widgets
Subject	Mobile App Development
Department	Information Technology
Aim	To understand and implement standard Flutter layout widgets: Container, GridView, ListView, S

Theory

Flutter provides powerful layout widgets to arrange and style UI elements. The widget tree is composed of nested widgets that define the visual structure.

Layout Widgets Overview

- Container – single-child box with padding, margin, color, and size
- GridView – 2D scrollable grid of widgets
- ListView – scrollable list of widgets (vertical or horizontal)
- Stack – overlapping widgets (one on top of another)
- Card – Material Design card with rounded corners and shadow
- ListTile – pre-designed list item with leading/trailing icon and text

Source Code

```
import 'package:flutter/material.dart'; void main() => runApp(MyApp()); class MyApp extends StatelessWidget { @override Widget build(BuildContext context) { return MaterialApp( home: MyHomePage(), ); } } class MyHomePage extends StatefulWidget { @override _MyHomePageState createState() => _MyHomePageState(); } class _MyHomePageState extends State<MyHomePage> { @override Widget build(BuildContext context) { return Scaffold( appBar: AppBar(title: Text('IT Department')), body: SafeArea( child: Container( height: 200.0, width: 200.0, color: Colors.green, child: Center( child: Text( 'Container Widget', style: TextStyle(color: Colors.white, fontSize: 18), ), ), ), ), ); } }
```

Expected Output

A green-colored box (200x200) displayed on the screen with the text "Container Widget" centered inside it, below an AppBar.

Conclusion

Container and other layout widgets provide the foundation for building structured UIs in Flutter. Understanding these widgets is essential for creating any Flutter application.

VIVA QUESTIONS & ANSWERS

Q: What is the Container widget?

A: Container is a convenience widget that combines common painting, positioning, and sizing. It can have color, padding, margin, border, and size constraints.

Q: What is GridView in Flutter?

A: GridView is a scrollable 2D grid of widgets. GridView.count allows you to specify the number of columns.

Q: What is the difference between ListView and Column?

A: Column does not scroll and throws overflow error if content exceeds screen. ListView is scrollable and handles overflow automatically.

Q: What is a Stack widget?

A: Stack allows widgets to overlap each other. The first child is at the bottom, and subsequent children are placed on top.

Q: What is a Card widget?

A: Card is a Material Design widget with slightly rounded corners and a drop shadow (elevation). It creates a 3D raised effect.

Q: What is BoxDecoration?

A: BoxDecoration is used with Container to apply color, border, borderRadius, boxShadow, and gradient styling.

EXPERIMENT 3: Flutter Form Widget

Experiment No.	3
Title	Form Widget & Input Validation in Flutter
Subject	Mobile App Development
Department	Information Technology
Aim	To create an interactive form using Flutter Form widget with input validation using TextFormFie

Theory

The Form widget provides a container for grouping and validating multiple form fields. It uses a GlobalKey to access the form state and call validation.

Form Components

- Form – container that groups form fields, uses GlobalKey
- TextFormField – text input with built-in validation support
- GlobalKey<FormState> – unique key to access form state methods
- FormState.validate() – triggers all validator functions
- FormState.save() – triggers all onSaved callbacks
- FormState.reset() – clears all form fields
- validator – function that returns null if valid, error string if invalid

Source Code

```
import 'package:flutter/material.dart'; void main() => runApp(MyApp()); class MyApp extends StatelessWidget { @override Widget build(BuildContext context) { return MaterialApp(home: MyForm()); } } class MyForm extends StatelessWidget { final _key = GlobalKey<FormState>(); @override Widget build(BuildContext context) { return Scaffold( appBar: AppBar(title: Text('Form Widget Demo')), body: Container( padding: EdgeInsets.all(32), child: Form( key: _key, child: Column( children: [ TextFormField( initialValue: 'User Name', decoration: InputDecoration(labelText: 'Username'), validator: (value) { if (value!.isEmpty) return 'Cannot be empty'; if (value.length <= 5) return 'Must be > 5 chars'; return null; }, ), SizedBox(height: 16), ElevatedButton( onPressed: () { if (_key.currentState!.validate()) { print('Form submitted successfully'); ScaffoldMessenger.of(context).showSnackBar( SnackBar(content: Text('Data Submitted!')), ); } }, child: Text('Submit'), ), ], ), ), ), ); } }
```

Expected Output

A form with a text field is displayed. When Submit is clicked, if the username is empty or shorter than 5 chars, an error message appears below the field. On valid input, a SnackBar appears.

Conclusion

The Form widget with TextFormField provides a complete and robust input validation mechanism. GlobalKey enables form state management for validation and submission.

VIVA QUESTIONS & ANSWERS

Q: What is the purpose of GlobalKey in Form?

A: GlobalKey<FormState> provides a reference to the form's state object, allowing us to call validate(), save(), and reset() on the form.

Q: What does the validator function return?

A: validator returns null if the input is valid, or a non-null String (error message) if validation fails.

Q: What is InputDecoration?

A: InputDecoration customizes the appearance of TextFormField – labelText, hintText, prefixIcon, suffixIcon, border, errorText.

Q: Difference between TextFormField and TextField?

A: TextFormField integrates with Form widget and supports validator, onSaved callbacks. TextField is a raw input widget without form integration.

Q: What is a SnackBar?

A: SnackBar is a brief message at the bottom of the screen, shown using ScaffoldMessenger.of(context).showSnackBar(). It auto-disappears after a duration.

EXPERIMENT 4: SafeArea, AppBar, Row & Column

Experiment No.	4
Title	SafeArea, AppBar, Row and Column Widgets
Subject	Mobile App Development
Department	Information Technology
Aim	To implement and understand SafeArea, AppBar, Row and Column widgets for building respon

Theory

SafeArea prevents UI overlap with system UI (notches, status bar). AppBar provides the top navigation bar. Row and Column are the primary layout widgets for arranging children.

Source Code

```
import 'package:flutter/material.dart'; void main() => runApp(MyApp()); class MyApp extends StatelessWidget { @override Widget build(BuildContext context) { return MaterialApp( home: Scaffold( appBar: AppBar( title: Text('SafeArea & Layout Demo'), backgroundColor: Colors.indigo, actions: [ IconButton(icon: Icon(Icons.search), onPressed: () {}), ], ), body: SafeArea( child: Column( mainAxisAlignment: MainAxisAlignment.center, children: [ Row( mainAxisAlignment: MainAxisAlignment.spaceEvenly, children: [ Icon(Icons.star, color: Colors.amber, size: 40), Column( children: [ Text('Flutter', style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold)), Text('Cross-Platform UI Framework'), ], ), Icon(Icons.flutter_dash, color: Colors.blue, size: 40), ], ), SizedBox(height: 20), Text('Body content is safely padded', textAlign: TextAlign.center), ], ), ), ), ); } }
```

Expected Output

An app with an indigo AppBar, SafeArea-padded content, a Row with icons and text, and a centered message below.

Conclusion

SafeArea ensures the app content is not obscured by system UI. AppBar provides standard navigation. Row and Column are fundamental layout primitives.

VIVA QUESTIONS & ANSWERS

Q: What is SafeArea and why is it used?

A: SafeArea adds inset padding to avoid system UI intrusions (notches, status bars, navigation bars). It makes the app adaptive to different devices.

Q: What properties does AppBar have?

A: title, leading, actions, backgroundColor, elevation, bottom, flexibleSpace, shape, centerTitle, toolbarHeight.

Q: What is CrossAxisAlignment?

A: CrossAxisAlignment aligns children perpendicular to the main axis. For Row it is vertical alignment; for Column it is horizontal alignment.

Q: What is the Expanded widget?

A: Expanded makes a child of Row/Column fill the available space along the main axis. It takes a flex factor (default 1).

Q: What is Flexible vs Expanded?

A: Expanded forces a child to fill available space. Flexible allows the child to be smaller than available space if it doesn't need all of it.

EXPERIMENT 5: Flutter Icons & Asset Images

Experiment No.	5
Title	Flutter Icons and Asset Images
Subject	Mobile App Development
Department	Information Technology
Aim	To use Flutter Icon widget and Image.asset() to display icons and images in a Flutter applicatio

Theory

Flutter provides a rich set of Material Design icons via the Icons class. Asset images are bundled with the app and loaded using Image.asset().

Adding Assets to pubspec.yaml

```
flutter: assets: - assets/images/photo.jpg - assets/images/ # includes all images in folder
```

Source Code – Icons

```
import 'package:flutter/material.dart'; void main() => runApp(MyApp()); class MyApp extends StatelessWidget { @override Widget build(BuildContext context) { return MaterialApp( home: Scaffold( appBar: AppBar(title: Text('Flutter Icons')), body: Row( mainAxisAlignment: MainAxisAlignment.spaceAround, children: [ Icon(Icons.camera_enhance, size: 50, color: Colors.blue), Icon(Icons.camera_front, size: 50, color: Colors.red), Icon(Icons.camera_rear, size: 50, color: Colors.green), ], ), ), ); } }
```

Source Code – Asset Image

```
// Q2: Display Image from Assets import 'package:flutter/material.dart'; void main() => runApp(MyApp()); class MyApp extends StatelessWidget { @override Widget build(BuildContext context) { return MaterialApp( home: Scaffold( appBar: AppBar(title: Text('Asset Image Demo')), body: Center( child: Column( mainAxisAlignment: MainAxisAlignment.center, children: [ Image.asset( 'assets/images/photo.jpg', width: 300, height: 200, fit: BoxFit.cover, ), SizedBox(height: 16), Text('Loaded from Assets'), ], ), ), ), ); } }
```

Expected Output

Experiment 5a: Three camera icons (blue, red, green) displayed in a row. Experiment 5b: An image loaded from assets displayed in the center of the screen.

Conclusion

Flutter's Icon widget provides easy access to Material icons. Image.asset() loads bundled images, while Image.network() loads remote images.

VIVA QUESTIONS & ANSWERS

Q: How do you add images to a Flutter project?

A: Create an assets/images folder, add images there, then register the path in pubspec.yaml under flutter: assets:

Q: What is Image.network() vs Image.asset()?

A: Image.asset() loads images from the app bundle. Image.network() loads from a URL. Network images require internet access.

Q: What is BoxFit in Flutter?

A: BoxFit controls how an image is fitted in its space: cover (fills, crops), contain (fits inside), fill (stretches), fitWidth, fitHeight.

Q: What is the Icon widget?

A: Icon displays a Material Design icon using the Icons class. Properties include size, color, and semanticLabel.

Q: How do you use custom fonts in Flutter?

A: Add font files to assets, register them in pubspec.yaml under flutter: fonts:, then use them in TextStyle(fontFamily: "FontName").

EXPERIMENT 6: Flutter Navigation & Gestures

Experiment No.	6
Title	Flutter Navigation, Named Routes & Gesture Handling
Subject	Mobile App Development
Department	Information Technology
Aim	To implement screen navigation using Navigator, named routes, and handle user gestures usin

Theory

Flutter uses a Navigator widget with a stack-based history to manage screen transitions. GestureDetector wraps widgets to detect touch input.

Source Code – Navigation

```
import 'package:flutter/material.dart'; void main() { runApp(MaterialApp( title: 'Flutter Navigation', theme: ThemeData(primarySwatch: Colors.green), home: FirstRoute(), )); } class FirstRoute extends StatelessWidget { @override Widget build(BuildContext context) { return Scaffold( appBar: AppBar(title: Text('First Screen')), body: Center( child: ElevatedButton( child: Text('Go to Second Screen'), onPressed: () { Navigator.push( context, MaterialPageRoute(builder: (context) => SecondRoute()), ); }, ), ), ); } } class SecondRoute extends StatelessWidget { @override Widget build(BuildContext context) { return Scaffold( appBar: AppBar(title: Text('Second Screen')), body: Center( child: ElevatedButton( child: Text('Go Back'), onPressed: () => Navigator.pop(context), ), ), ); } }
```

Named Routes

```
// Define routes in MaterialApp MaterialApp( routes: { '/': (context) => HomeScreen(), '/details': (context) => DetailsScreen(), '/settings': (context) => SettingsScreen(), }, initialRoute: '/', ) // Navigate using named route Navigator.pushNamed(context, '/details'); // Go back Navigator.pop(context);
```

Gesture Detection

```
GestureDetector( onTap: () => print('Tapped!'), onDoubleTap: () => print('Double tapped!'), onLongPress: () => print('Long pressed!'), onHorizontalDragUpdate: (details) => print('Swiped: ${details.delta}'), child: Container( width: 200, height: 100, color: Colors.blue, child: Center(child: Text('Touch Me', style: TextStyle(color: Colors.white))), ), )
```

Conclusion

Navigator.push() adds a new screen to the stack; Navigator.pop() removes it. Named routes provide cleaner navigation for multi-screen apps. GestureDetector enables custom touch interactions.

VIVA QUESTIONS & ANSWERS

Q: What is Navigator in Flutter?

A: Navigator manages a stack of Route objects. push() adds a new screen; pop() removes the current screen and returns to the previous one.

Q: What is MaterialPageRoute?

A: MaterialPageRoute creates a route with Material Design page transitions. It takes a builder function that returns the destination widget.

Q: What are named routes?

A: Named routes use string identifiers (e.g., "/home") defined in MaterialApp routes map, enabling navigation with Navigator.pushNamed(context, "/home").

Q: What is GestureDetector?

A: GestureDetector is a non-visual widget that detects gestures (tap, double-tap, long press, drag, swipe, pinch) on its child widget.

Q: Difference between Navigator.push and Navigator.pushReplacement?

A: push adds a new route on top. pushReplacement replaces the current route, removing it from the stack (no back button to previous screen).

Q: What gestures does Flutter support?

A: Tap, DoubleTap, LongPress, VerticalDrag, HorizontalDrag, Pan, Scale (pinch to zoom), and ForcePressure.

EXPERIMENT 7: PWA – Web App Manifest

Experiment No.	7
Title	Progressive Web App – Web App Manifest (manifest.json)
Subject	Mobile App Development
Department	Information Technology
Aim	To create and configure a Web App Manifest (manifest.json) file to make a web application inst

Theory

A Web App Manifest is a JSON file that tells the browser about the web application, enabling it to be installed on the user's home screen like a native app. It is a key component of Progressive Web Apps (PWA).

Manifest Key Properties

- name – full name of the app (shown during install)
- short_name – short name for home screen / launcher
- start_url – URL that loads when app is launched
- display – UI mode: standalone, fullscreen, minimal-ui, browser
- background_color – splash screen background color
- theme_color – browser toolbar color
- icons – array of app icons for different screen sizes
- orientation – preferred screen orientation (portrait/landscape)

Source Code – manifest.json

```
{ "name": "My Ecommerce App", "short_name": "ecommerce", "start_url": "/ecommerce.html",
"display": "standalone", "background_color": "#FFE9D2", "theme_color": "#FFE1C4",
"orientation": "portrait-primary", "icons": [ { "src": "/img/icons/icon-72x72.png",
"type": "image/png", "sizes": "72x72" }, { "src": "/img/icons/icon-96x96.png", "type":
"image/png", "sizes": "96x96" }, { "src": "/img/icons/icon-128x128.png","type":
"image/png", "sizes": "128x128"}, { "src": "/img/icons/icon-192x192.png","type":
"image/png", "sizes": "192x192"}, { "src": "/img/icons/icon-512x512.png","type":
"image/png", "sizes": "512x512"} ] }
```

How to Link Manifest in HTML

```
<head> <link rel="manifest" href="/manifest.json"> <meta name="theme-color"
content="#FFE1C4"> </head>
```

Expected Output

The web app becomes installable. Users see an "Add to Home Screen" prompt in the browser. After installation, the app launches in standalone mode (no browser chrome) with the specified splash screen and icon.

Conclusion

The manifest.json file is essential for making a web app a PWA. It provides metadata for installation, customizes the splash screen, and improves the user experience by making the app feel native.

VIVA QUESTIONS & ANSWERS

Q: What is a PWA?

A: Progressive Web App is a web app that uses modern APIs to deliver native app-like experiences including offline access, push notifications, and home screen installation.

Q: What is manifest.json?

A: manifest.json is a JSON configuration file that tells browsers about the PWA – its name, icons, start URL, display mode, and theme colors.

Q: What does display: standalone do?

A: standalone makes the app launch without browser UI (no address bar, no browser toolbar), giving a native app-like appearance.

Q: Why are multiple icon sizes needed?

A: Different devices and contexts require different icon resolutions (72x72 for low-res, 192x192 for home screen, 512x512 for splash screen).

Q: What are the 3 pillars of PWA?

A: Reliable (works offline via Service Worker), Fast (loads quickly), and Engaging (installable, push notifications, full-screen).

EXPERIMENT 8: Service Worker Lifecycle

Experiment No.	8
Title	Service Worker – Registration, Lifecycle & Basic Events
Subject	Mobile App Development
Department	Information Technology
Aim	To understand and implement Service Worker registration and its lifecycle events (install, activ

Theory

A Service Worker is a JavaScript file that runs in the background, separate from the main browser thread. It acts as a proxy between the web app and the network, enabling offline functionality and caching.

Service Worker Lifecycle

- 1. Registration – Browser downloads & registers the SW file
- 2. Installation – install event fires; SW precaches assets
- 3. Waiting – new SW waits for old pages to close
- 4. Activation – activate event fires; old caches are cleared
- 5. Idle – SW is ready to handle fetch/push events
- 6. Terminated – browser may terminate SW when idle to save memory

Source Code – service-worker.js

```
// Register Service Worker (in main JS) if ('serviceWorker' in navigator) {
navigator.serviceWorker.register('/service-worker.js') .then(reg => console.log('SW
registered:', reg.scope)) .catch(err => console.log('SW failed:', err)); } //
service-worker.js const CACHE_NAME = 'app-v1'; // Install Event
self.addEventListener('install', evt => { console.log('Service Worker: Installed');
evt.waitUntil( caches.open(CACHE_NAME).then(cache => { return
cache.addAll(['/index.html', '/style.css', '/app.js']); }) ); }); // Activate Event
self.addEventListener('activate', evt => { console.log('Service Worker: Activated');
evt.waitUntil( caches.keys().then(keys => { return Promise.all( keys.filter(key => key
!== CACHE_NAME).map(key => caches.delete(key)) ); }) ); }); // Fetch Event
self.addEventListener('fetch', evt => { console.log('Service Worker: Fetching',
evt.request.url); evt.respondWith( caches.match(evt.request).then(response => { return
response || fetch(evt.request); }) ); });
```

Expected Output

In DevTools > Application > Service Workers: Status shows "activated and running". Console logs "Service Worker: Installed" and "Service Worker: Activated". The app loads from cache when offline.

Conclusion

Service Workers enable offline-first web applications by intercepting network requests and serving cached responses. Understanding the lifecycle is essential for implementing robust PWAs.

VIVA QUESTIONS & ANSWERS

Q: What is a Service Worker?

A: A Service Worker is a JavaScript script that runs in the browser background, separate from the main thread. It intercepts network requests and enables offline functionality.

Q: What events does a Service Worker have?

A: install (first install/update), activate (after install when old pages close), fetch (every network request), push (push notifications), sync (background sync).

Q: What is evt.waitUntil()?

A: evt.waitUntil() extends the lifetime of the event until the passed Promise resolves. Used in install/activate to ensure caching completes before SW activates.

Q: What is the scope of a Service Worker?

A: Scope defines which URLs the SW controls. By default, the scope is the directory of the SW file. All pages within scope are controlled by the SW.

Q: Can Service Workers access the DOM?

A: No. Service Workers run in a separate thread and cannot access the DOM. They communicate with pages using the postMessage API.

EXPERIMENT 9: Static Caching in Service Worker

Experiment No.	9
Title	Static (Pre-Cache) Caching in Service Worker
Subject	Mobile App Development
Department	Information Technology
Aim	To implement static caching in a Service Worker file by pre-caching specified assets during the

Theory

Static caching (pre-caching) stores specific files in the browser cache during Service Worker installation. These cached files are then served without network requests, enabling offline access.

Static Caching Strategy

- Pre-cache assets during the install event
- All specified assets are cached before SW activates
- If any asset fails to cache, the entire install fails
- Best for: HTML, CSS, JS, images that rarely change
- Limitation: Cache must be manually updated when files change
- Cache versioning ensures users get updated content

Source Code – Static Caching

```
const staticCacheName = 'site-static-v1'; const assets = [ '/index.html', '/style.css', '/app.js', '/offline.html', '/img/logo.png', ]; // Install: Pre-cache assets self.addEventListener('install', evt => { evt.waitUntil( caches.open(staticCacheName).then(cache => { console.log('Caching static assets...'); return cache.addAll(assets); }) ); }); // Activate: Delete old caches self.addEventListener('activate', evt => { evt.waitUntil( caches.keys().then(keys => { return Promise.all( keys .filter(key => key !== staticCacheName) .map(key => { console.log('Deleting old cache:', key); return caches.delete(key); }) ); }) ); }); // Fetch: Serve from cache first, fallback to network self.addEventListener('fetch', evt => { evt.respondWith( caches.match(evt.request).then(cacheRes => { return cacheRes || fetch(evt.request) .catch(() => caches.match('/offline.html')); }) ); });
```

Expected Output

When offline, the app loads from cache instead of showing a "No Internet" error. In DevTools > Application > Cache Storage: the staticCacheName cache contains all pre-cached files.

Conclusion

Static caching is the simplest and most reliable caching strategy. It guarantees critical assets are always available offline. Cache versioning ensures stale content is removed when the SW updates.

VIVA QUESTIONS & ANSWERS

Q: What is static/pre-caching?

A: Static caching pre-downloads and stores specific assets (HTML, CSS, JS, images) during the Service Worker install event, making them available offline.

Q: What is cache versioning?

A: Cache versioning (site-static-v1, v2, etc.) ensures that when SW updates, the old cache is deleted and new assets are cached, preventing stale content.

Q: What happens if one asset fails to cache in addAll()?

A: If any asset in cache.addAll() fails (404, network error), the entire install fails and the SW does not activate. Use individual cache.add() for optional assets.

Q: What is Cache-First strategy?

A: Cache-First checks the cache first; if found, serves from cache (fast). Only falls back to network if not cached. Used for static assets.

Q: What is the difference between cache.add() and cache.put()?

A: cache.add() fetches the URL and stores it. cache.put() stores a response object directly without fetching. cache.addAll() is add() for multiple URLs.

EXPERIMENT 10: Dynamic Caching in Service Worker

Experiment No.	10
Title	Dynamic Caching in Service Worker
Subject	Mobile App Development
Department	Information Technology
Aim	To implement dynamic caching in a Service Worker that caches resources at runtime as they a

Theory

Dynamic caching stores responses as they are fetched at runtime. Unlike static caching, only requested resources are cached. This is more flexible and efficient for large apps with many assets.

Dynamic Caching Process

- 1. User requests a page/resource
- 2. SW intercepts the fetch event
- 3. SW checks cache – if found, serve from cache (fast!)
- 4. If not in cache: fetch from network
- 5. Clone the network response
- 6. Store the response clone in dynamic cache
- 7. Return original response to the page

Source Code – Dynamic Caching

```
const staticCacheName = 'site-static-v1'; const dynamicCacheName = 'site-dynamic-v1';
const staticAssets = ['/index.html', '/style.css', '/app.js']; // Install: pre-cache
static assets self.addEventListener('install', evt => { evt.waitUntil(
caches.open(staticCacheName).then(cache => cache.addAll(staticAssets)) ); }); //
Activate: cleanup old caches self.addEventListener('activate', evt => { evt.waitUntil(
caches.keys().then(keys => Promise.all( keys.filter(k => k !== staticCacheName && k !==
dynamicCacheName) .map(k => caches.delete(k)) ) ) ); }); // Fetch: Cache-First with
Dynamic Caching fallback self.addEventListener('fetch', evt => { evt.respondWith(
caches.match(evt.request).then(cacheRes => { // Return cached response if available
return cacheRes || fetch(evt.request).then(fetchRes => { // Cache the new response
dynamically return caches.open(dynamicCacheName).then(cache => {
cache.put(evt.request.url, fetchRes.clone()); return fetchRes; // Return original
response }); }); }) ); });
```

Static vs Dynamic Caching Comparison

Aspect	Static Caching	Dynamic Caching
When cached	During install event	At runtime (on first request)
What is cached	Pre-defined list of assets	Any resource that is requested
Storage use	Known size	Grows over time
Best for	App shell, critical assets	API data, dynamic content
Cache grows?	No – fixed at install	Yes – grows with usage

Expected Output

New pages are served from network on first visit and from dynamic cache on subsequent visits. DevTools > Cache Storage shows the dynamicCacheName cache growing as pages are visited.

Conclusion

Dynamic caching complements static caching by automatically caching resources at runtime. Combining both strategies creates a robust offline-first PWA experience.

VIVA QUESTIONS & ANSWERS

Q: What is dynamic caching?

A: Dynamic caching stores responses in cache as they are fetched at runtime. Resources are cached only when they are requested, not pre-downloaded.

Q: Why clone the response in dynamic caching?

A: A Response object is a stream and can only be consumed once. Cloning (fetchRes.clone()) allows storing one copy in cache while returning the original to the page.

Q: What is the Network-First strategy?

A: Network-First fetches from network first; if successful, caches the response. Falls back to cache if network fails. Good for frequently updated content.

Q: What are the main caching strategies?

A: Cache-First (cache > network), Network-First (network > cache), Stale-While-Revalidate (cache instantly, update in background), Cache-Only, Network-Only.

Q: How do you limit dynamic cache size?

A: By implementing a cache size limit function that deletes the oldest entries when cache exceeds a maximum number of items.

Q: What is cache.put() vs cache.add()?

A: cache.put(request, response) stores a response object directly. cache.add(url) fetches the URL then stores the response automatically.

PART 4: QUICK REVISION

Key Concepts, Keywords & Summary for Last-Minute Prep

Dart Programming – Quick Reference

Keyword	Meaning
`var`	Type-inferred variable
`final`	Set once at runtime, immutable
`const`	Compile-time constant
`late`	Non-null, initialized later
`dynamic`	Type can change at runtime
`async/await`	Asynchronous programming
`Future`	Single async value
`Stream`	Sequence of async values
`extends`	Class inheritance
`implements`	Implement an interface
`mixin / with`	Code reuse without inheritance
`abstract`	Class cannot be instantiated
`override`	Redefine parent method
`super`	Reference to parent class
`is / is!`	Type checking
`as`	Type casting
`??`	Null coalescing – use if null
`?.`	Null-safe member access
`...`	Spread operator for collections
`yield`	Emit value from generator
`sealed`	Closed class hierarchy (Dart 3)

Flutter – Quick Reference

Widget / Method	Purpose
`runApp()`	Entry point – attaches root widget to screen

`MaterialApp`	Root widget for Material Design apps
`Scaffold`	Page structure: AppBar, Body, FAB, Drawer
`StatelessWidget`	Immutable widget – no state changes
`StatefulWidget`	Widget with mutable state
`setState()`	Marks state dirty; triggers rebuild
`BuildContext`	Location of widget in tree
`Column / Row`	Vertical / Horizontal layout
`Container`	Box with padding, margin, color, decoration
`Text / RichText`	Display single or multi-style text
`Image.asset()`	Load image from bundled assets
`Image.network()`	Load image from URL
`Navigator.push()`	Navigate to new screen
`Navigator.pop()`	Go back to previous screen
`GestureDetector`	Detect touch gestures
`Form + TextFormField`	Input forms with validation
`SafeArea`	Avoid system UI intrusions
`AppBar`	Top navigation bar
`initState()`	Called once on widget creation
`dispose()`	Cleanup when widget destroyed
`MediaQuery`	Get screen dimensions & orientation
`Theme.of(context)`	Access app theme

TOP 50 VIVA QUESTIONS

Most Expected Questions for Exam & Viva

#	Question	Answer
1	**What is Flutter?**	Open-source UI framework by Google for cross-platform apps using Dart.
2	**What language does Flutter use?**	Dart – a strongly typed, object-oriented language by Google.
3	**What is runApp()?**	Entry point function that inflates root widget and attaches to screen.
4	**What is a Widget?**	Everything in Flutter is a widget – the basic building block of UI.
5	**Stateless vs Stateful Widget?**	StatelessWidget has no mutable state; StatefulWidget has State class with setState().
6	**What is setState()?**	Marks state as dirty and triggers a widget rebuild.
7	**What is BuildContext?**	Reference to widget's location in widget tree; used to access theme/navigation.
8	**What is the widget tree?**	Hierarchical structure of all widgets from root to leaf in a Flutter app.
9	**What is initState() used for?**	Initialize resources, controllers, and data on first widget creation.
10	**What is dispose()?**	Cleanup method called when widget is permanently removed; prevents memory leaks.
11	**What is var in Dart?**	Type-inferred variable; type is set on first assignment and cannot change.
12	**final vs const?**	final set once at runtime; const is compile-time constant.
13	**What is null safety?**	Ensures variables cannot be null unless explicitly declared with ?.
14	**What are mixins?**	Code reuse mechanism using with keyword; provides multiple inheritance-like behavior.

15	**What is async/await?**	Asynchronous programming: async marks function, await waits for Future to complete.
16	**Future vs Stream?**	Future gives one value asynchronously; Stream gives multiple values over time.
17	**What is a sealed class?**	Class whose hierarchy is restricted to same library; enables exhaustive switch.
18	**What is Navigator?**	Stack-based widget for managing routes (screens) in Flutter app.
19	**What is MaterialPageRoute?**	Creates a route with Material Design transition animation.
20	**What is Named Routes?**	Routes identified by string names defined in MaterialApp routes map.
21	**What is GestureDetector?**	Non-visual widget that detects touch gestures on its child.
22	**What is SafeArea?**	Adds padding to prevent UI from being obscured by notches/status bars.
23	**What is a Container?**	Versatile layout widget with padding, margin, color, size, decoration.
24	**Row vs Column?**	Row = horizontal layout; Column = vertical layout.
25	**What is Expanded?**	Makes child of Row/Column fill available space along main axis.
26	**What is a Form widget?**	Groups form fields and manages validation using GlobalKey.
27	**What is validator?**	Function in TextFormField that returns null if valid, error string if invalid.
28	**What is GlobalKey?**	Unique key that allows accessing a widget's state from anywhere.
29	**What is pubspec.yaml?**	Configuration file for Flutter project: declares dependencies and assets.
30	**What is flutter doctor?**	Command to check Flutter installation and identify missing dependencies.
31	**What is Hot Reload?**	Instantly injects code changes without restarting; preserves app state.

32	**What is BoxFit?**	How images are fitted: cover (fill+crop), contain (fit inside), fill (stretch).
33	**What is Image.asset()?**	Loads image from the app's bundled assets declared in pubspec.yaml.
34	**What is a PWA?**	Progressive Web App – web app with native features: offline, installable, notifications.
35	**What is manifest.json?**	JSON file declaring PWA metadata: name, icons, start URL, display mode.
36	**What is a Service Worker?**	JavaScript script running in background; intercepts network requests for offline support.
37	**SW lifecycle stages?**	Registration → Installation (install event) → Activation (activate event) → Fetch events.
38	**What is static caching?**	Pre-caching specific assets during SW install event for guaranteed offline availability.
39	**What is dynamic caching?**	Caching resources at runtime as they are fetched; cache grows with usage.
40	**Why clone response in SW?**	Response is a stream consumed only once; clone() allows storing copy while returning original.
41	**What is evt.waitUntil()?**	Extends SW event lifetime until the passed Promise resolves.
42	**Cache-First strategy?**	Serve from cache if available; fallback to network. Used for static assets.
43	**What are Isolates in Dart?**	Independent execution units with separate memory; communicate via message passing.
44	**What is abstract class?**	Class that cannot be instantiated; defines abstract methods subclasses must implement.
45	**What is @override?**	Annotation indicating a method intentionally overrides a parent class method.
46	**What is extends vs implements?**	extends inherits implementation; implements enforces contract without code sharing.
47	**What are Records in Dart?**	Anonymous, immutable aggregate types introduced in Dart 3.0 for grouping values.
48	**What are extension methods?**	Add functionality to existing types without subclassing or modifying the original class.

49	**What is a factory constructor?**	Constructor that can return existing instance; used for singleton and caching patterns.
50	**What is the ?? operator?**	Null coalescing: returns left side if not null, otherwise returns right side.

BEST OF LUCK FOR YOUR VIVA!